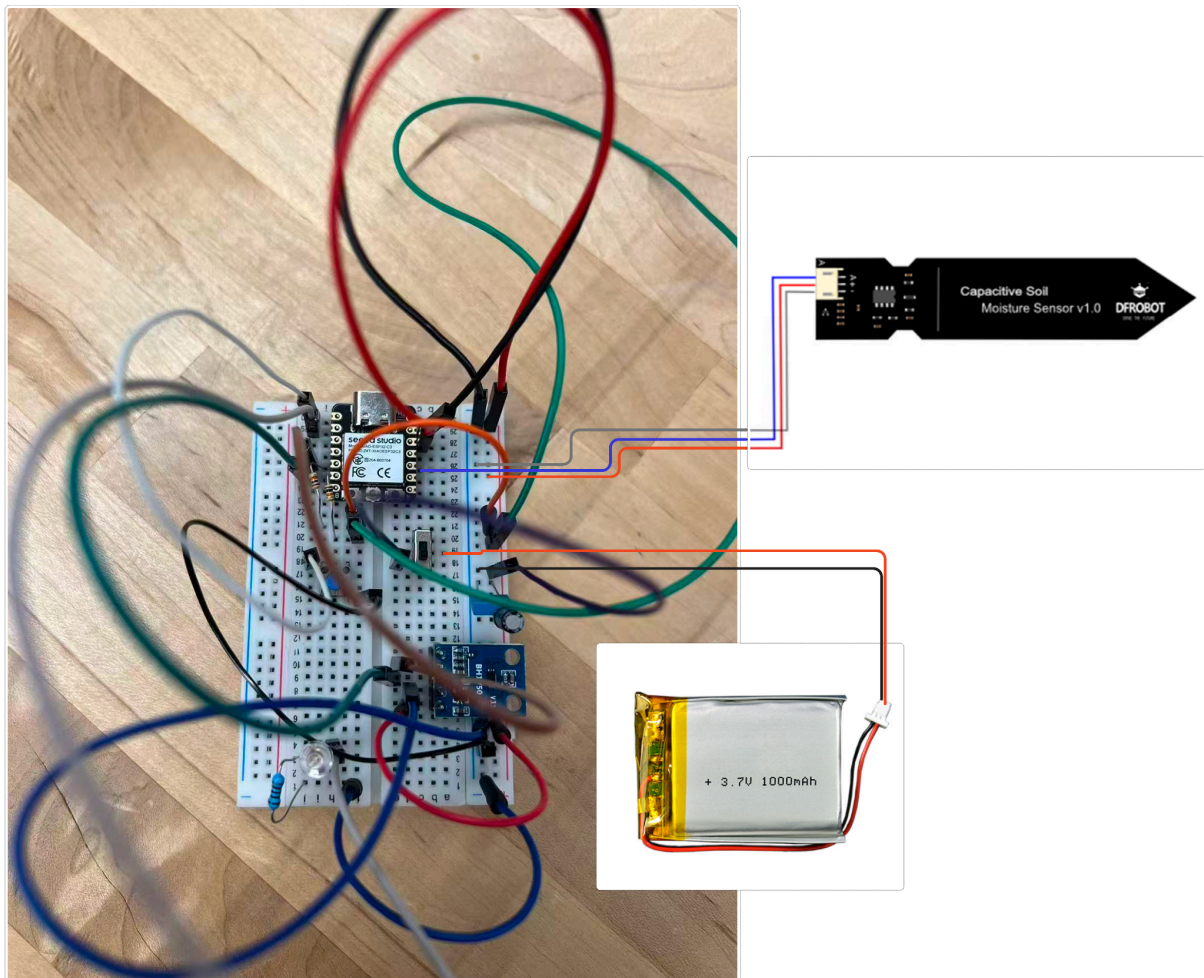


Second Device

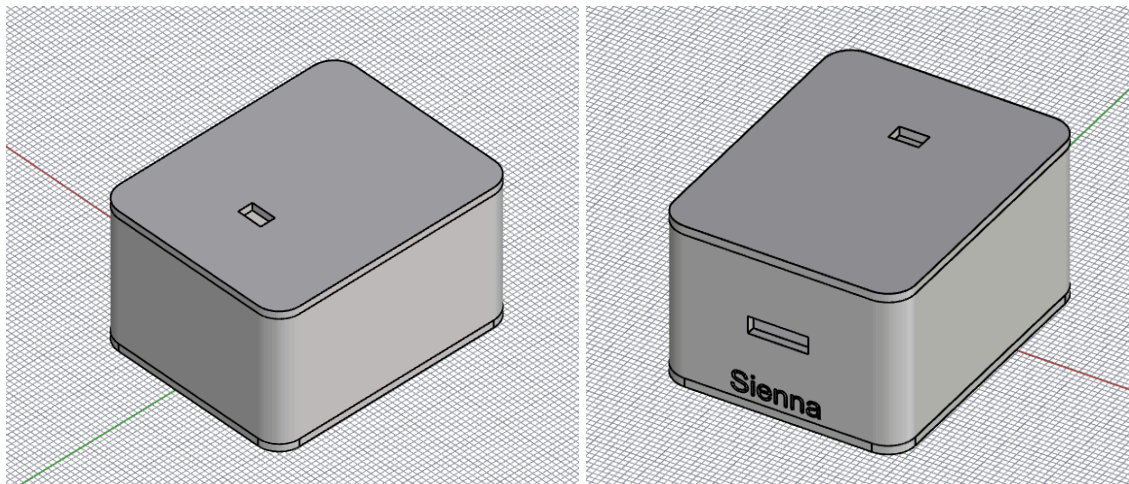
Description

The Sensor Device collects real-time environmental data to evaluate plant health. It measures ambient light intensity using a BH1750 light sensor (I2C) and soil moisture using an analog capacitive moisture sensor. The ESP32-C3 processes these inputs, calculates a health score (0–100), and transmits the result wirelessly via BLE. This score is received by the Display Device, which converts it into a physical needle position to visually indicate the plant's overall health condition.

One Photo of Functional Breadboard



Two screenshots of CAD Model



Link to code in Github

https://github.com/Sienna-Zhang/514_final/tree/main/514_final/code/sensor

Functional Code of Second Device

```
#include <Arduino.h>
#include <Wire.h>
#include <BH1750.h>
#include <NimBLEDevice.h>

// ----- Pin mapping (from your schematic) -----
static const int PIN_I2C_SDA = 4; // GPIO4_A3_D3_SDA
static const int PIN_I2C_SCL = 6; // GPIO6_A5_D5_SCL
static const int PIN_MOIST_A = 2; // GPIO2_A1_D1 <- SEN0193 AOUT
static const int PIN_LED = 1; // change if your LED is on another GPIO

// ----- Moisture calibration (YOU SHOULD CALIBRATE) -----
// Read ADC when sensor is in water-saturated soil (wet) and in air (dry).
// Capacitive sensors often: dry ADC higher, wet ADC lower. If yours is opposite, swap.
static const int MOIST_WET_ADC = 1200;
static const int MOIST_DRY_ADC = 3200;

// ----- Health weighting -----
static const float W_MOIST = 0.60f;
static const float W_LIGHT = 0.40f;

// ----- BLE UUIDs -----
static const char* BLE_DEVICE_NAME = "PlantSensor";
static const char* SVC_UUID = "6e400001-b5a3-f393-e0a9-e50e24dcca9e";
static const char* CH_UUID = "6e400002-b5a3-f393-e0a9-e50e24dcca9e"; // Notify: uint8
score
```

```

BH1750 lightMeter;

NimBLEServer* pServer = nullptr;
NimBLECharacteristic* pCharScore = nullptr;

static uint8_t lastScore = 255;

static float clamp01(float x) {
    if (x < 0.0f) return 0.0f;
    if (x > 1.0f) return 1.0f;
    return x;
}

static float map01(int v, int inMin, int inMax) {
    if (inMax == inMin) return 0.0f;
    float t = (float)(v - inMin) / (float)(inMax - inMin);
    return clamp01(t);
}

// Light factor: ramp up -> optimal -> slightly penalize too-bright
static float lightFactorFromLux(float lux) {
    // 0 at <=50 lux, 1 at 300 lux
    if (lux <= 50.0f) return 0.0f;
    if (lux < 300.0f) return (lux - 50.0f) / (300.0f - 50.0f);

    // keep 1 between 300 and 1500 lux
    if (lux <= 1500.0f) return 1.0f;

    // from 1500 to 5000 lux, drop from 1.0 to 0.6
    if (lux < 5000.0f) {
        float t = (lux - 1500.0f) / (5000.0f - 1500.0f);
        return 1.0f - 0.4f * t;
    }
    return 0.6f;
}

static uint8_t computeHealthScore(float lux, int moistAdc) {
    // Moist factor: 1 = wet, 0 = dry (assuming dry ADC higher)
    float moist01 = map01(moistAdc, MOIST_WET_ADC, MOIST_DRY_ADC);
    float moistFactor = 1.0f - moist01;

    float lightFactor = lightFactorFromLux(lux);

    float score01 = W_MOIST * moistFactor + W_LIGHT * lightFactor;
    int score = (int)roundf(100.0f * clamp01(score01));
    if (score < 0) score = 0;
    if (score > 100) score = 100;
    return (uint8_t)score;
}

```

```

}

class ServerCallbacks : public NimBLEServerCallbacks {
    void onConnect(NimBLEServer* s) override {
        digitalWrite(PIN_LED, HIGH);
    }
    void onDisconnect(NimBLEServer* s) override {
        digitalWrite(PIN_LED, LOW);
        NimBLEDevice::startAdvertising();
    }
};

static void setupBLE() {
    NimBLEDevice::init(BLE_DEVICE_NAME);
    NimBLEDevice::setPower(ESP_PWR_LVL_P9); // max

    pServer = NimBLEDevice::createServer();
    pServer->setCallbacks(new ServerCallbacks());

    NimBLEService* svc = pServer->createService(SVC_UUID);

    pCharScore = svc->createCharacteristic(
        CH_UUID,
        NIMBLE_PROPERTY::READ | NIMBLE_PROPERTY::NOTIFY
    );

    uint8_t initVal = 0;
    pCharScore->setValue(&initVal, 1);

    svc->start();

    NimBLEAdvertising* adv = NimBLEDevice::getAdvertising();
    adv->addServiceUUID(SVC_UUID);
    adv->setScanResponse(true);
    adv->start();
}

void setup() {
    pinMode(PIN_LED, OUTPUT);
    digitalWrite(PIN_LED, LOW);

    analogReadResolution(12); // ESP32 ADC default
    Wire.begin(PIN_I2C_SDA, PIN_I2C_SCL);

    // BH1750
    if (!lightMeter.begin(BH1750::CONTINUOUS_HIGH_RES_MODE)) {
        // still continue; you can debug with Serial if needed
    }
}

```

```
    setupBLE();  
}  
  
void loop() {  
    // Read sensors  
    float lux = lightMeter.readLightLevel();  
    int moistAdc = analogRead(PIN_MOIST_A);  
  
    uint8_t score = computeHealthScore(lux, moistAdc);  
  
    // Notify if changed or every cycle  
    if (score != lastScore) {  
        pCharScore->setValue(&score, 1);  
        pCharScore->notify();  
        lastScore = score;  
    }  
  
    delay(1200);  
}
```